



INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY

ASSIGNMENT

Assignment Title: Digital Forensics Case Handling, Autopsy and Sleuth Kit Analysis

Student Name: Ahmad Zubairu Garba

Student ID: 2025/FWSD/11463

Course Title: Basic Computer Skills for Digital Forensics

Course Code: SBT-DF202

Instructor: Aminu Idris

August 27, 2026

SECTION A: THEORETICAL OVERVIEW

1. DIGITAL INVESTIGATION PROCEDURES & EVIDENCE HANDLING

Digital forensics investigation relies on strict procedural controls to ensure evidence admissibility in a court of law:

Identification: Locating physical and digital media that may contain potential evidence.

Preservation: Isolating evidence to prevent alteration or contamination. The primary rule is to **never analyze original evidence**; work exclusively on bit-stream forensic copies.

Acquisition: Creating bit-stream copies of original storage media while calculating cryptographic hashes (MD5) to verify data integrity.

Chain of Custody: Documenting every individual who handled the evidence, the exact timestamp of transfer, storage conditions, and verification hash values.

2. THE SLEUTH KIT (TSK) ARCHITECTURE

The Sleuth Kit (TSK) is a C library and collection of command-line tools designed for file system and media analysis. TSK organizes operations across distinct functional layers:

- **Disk Image Layer** (`img_stat`, `img_cat`): Processes raw and container image formats.
- **Volume System Layer** (`mmls`, `mmstat`): Analyzes volume structures and partition tables (e.g., DOS, GPT).
- **File System Layer** (`fsstat`): Displays file system metadata, cluster sizes, layout, and structural boundaries.
- **Metadata Layer** (`istat`, `icat`, `ils`): Analyzes structural metadata pointers (Inodes in UNIX/Linux, MFT entries in NTFS, Directory Entries in FAT).
- **Data Unit Layer** (`blkcat`, `blkls`, `blkstat`): Reads raw clusters and sectors directly.
- **File Name Layer** (`fls`, `ffind`): Maps file and directory names to their corresponding metadata addresses.

SECTION B: PRACTICAL LAB EXECUTION & EVIDENTIARY ANALYSIS

TASK 1: WORKSPACE CREATION & EVIDENCE VERIFICATION

PART A: WORKSPACE PREPARATION

To maintain forensic isolation and adhere to evidence handling guidelines, a dedicated lab directory structure was created before beginning analysis.

The command `mkdir usb` created a dedicated working directory. The directory was accessed using `cd usb`, and the authorized forensic image `Ch01InChap01.dd` was downloaded using `wget`.

```
(garba@kali)-[~]
$ mkdir usb
(garba@kali)-[~]
$ cd usb
(garba@kali)-[~/usb]
$ wget -q https://www.dropbox.com/s/nw23q14vzsykyup/Ch01InChap01.dd
```

PART B: INITIAL EVIDENCE VERIFICATION AND HASHING

Before performing any forensic recovery, the file size and MD5 checksum of `Ch01InChap01.dd` were computed to record baseline evidence integrity.

```
(garba@kali)-[~/usb]
$ wget -q https://www.dropbox.com/s/nw23q14vzsykyup/Ch01InChap01.dd
(garba@kali)-[~/usb]
$ ls Ch01InChap01.dd -l
-rw-rw-r-- 1 garba garba 1474560 Aug 27 20:38 Ch01InChap01.dd
(garba@kali)-[~/usb]
$ md5sum Ch01InChap01.dd
a117773bcf1fc88ec0ab8e0a349fbbcb Ch01InChap01.dd
(garba@kali)-[~/usb]
$
```

The command `ls Ch01InChap01.dd -l` verified the exact byte size, and `md5sum Ch01InChap01.dd` generated the cryptographic hash.

MD5 Verification Hash: `a117773bcf1fc88ec0ab8e0a349fbbcb`

PART C: MINI CHAIN-OF-CUSTODY WORKSHEET

Field	Evidence Entry
Case Title	Digital Forensics Practical Lab 1
Item ID	Item 001
Evidence Description	Raw Disk Image (Ch01InChap01.dd)
File Size	1474560 bytes (1.5 MB)
Storage Location	/home/garba/usb/Ch01InChap01.dd
Verification MD5	a117773bcf1fc88ec0ab8e0a349fbbcb
Verification SHA-256	3ce8053e4f3d9c8ab98b3aadb2480685efb8e4980d34297b83bd5a09b1a7b122
Acquired By	Ahmad Zubairu Garba
Acquisition Date	August 27, 2026

TASK 2: SLEUTH KIT FILE SYSTEM & IMAGE METADATA ANALYSIS

PART A: IMAGE METADATA EXAMINATION (img_stat)

img_stat was executed to inspect the container image format properties.

```
(garba@kali)~[/usb]
$ img_stat Ch01InChap01.dd
IMAGE FILE INFORMATION

Image Type: raw
Size in bytes: 1474560
Sector size: 512

(garba@kali)~[/usb]
$ fsstat Ch01InChap01.dd
FILE SYSTEM INFORMATION

File System Type: FAT12

OEM Name: , 'k0nIHC
Volume ID: 0x0
Volume Label (Boot Sector):
Volume Label (Root Directory):
File System Type Label: *3*+M|

Sectors before file system: 0
File System Layout (in sectors)
```

The command `img_stat Ch01InChap01.dd` displayed the image format as raw, confirming a sector size of 512 bytes and total size of 1474560 bytes.

PART B: FILE SYSTEM DETAILS (fsstat)

`fsstat` was executed to display the file system layout and volume metrics.

```
(garba@kali)~[/usb]
$ img_stat Ch01InChap01.dd
IMAGE FILE INFORMATION

Image Type: raw
Size in bytes: 1474560
Sector size: 512

(garba@kali)~[/usb]
$ fsstat Ch01InChap01.dd
FILE SYSTEM INFORMATION

File System Type: FAT12

OEM Name: , 'k0nIHC
Volume ID: 0x0
Volume Label (Boot Sector):
Volume Label (Root Directory):
File System Type Label: *3*+M|

Sectors before file system: 0
File System Layout (in sectors)
```

The command `fsstat Ch01InChap01.dd` identified the file system as **FAT12**. It showed that the data area spans sectors 19–2879, the root directory occupies sectors 19–32, and the cluster area spans clusters 2 through 2848.

PART C: DIRECTORY LISTING & DELETED ENTRIES (fls)

Directory structures and deleted files were enumerated using `fls` and `fls -d`.

```
(garba@kali)-[~/usb]
$ fls -d Ch01InChap01.dd
r/r * 8:      Billing Letter.doc
r/r * 11:     confirmation.txt
r/r * 15:     letter1.txt
r/r * 17:     Regrets.doc

(garba@kali)-[~/usb]
$ fls Ch01InChap01.dd
r/r 5: Client Info.mdb
r/r * 8:      Billing Letter.doc
r/r * 11:     confirmation.txt
r/r 13: Income.xls
r/r * 15:     letter1.txt
r/r * 17:     Regrets.doc
v/v 45779: $MBR
v/v 45780: $FAT1
v/v 45781: $FAT2
V/V 45782: $OrphanFiles

(garba@kali)-[~/usb]
$
```

The command `fls Ch01InChap01.dd` listed all directory entries, revealing target file `Income.xls` at metadata address (inode) **13**. The command `fls -d Ch01InChap01.dd` filtered the directory to display deleted entries marked with `*`, identifying deleted files such as `Billing Letter.doc` (inode 8), `confirmation.txt` (inode 11), `letter1.txt` (inode 15), and `Regrets.doc` (inode 17).

TASK 3: RECOVERY AND METADATA EXAMINATION OF DELETED EVIDENCE

PART A: DELETED FILE EXTRACTION VIA INODE (`icat`)

`icat` was used to recover deleted file contents using metadata pointers.

```
(garba@kali)-[~/usb]
$ icat Ch01InChap01.dd 15 > letter1.txt

(garba@kali)-[~/usb]
$ cat letter1.txt
Earl,
We need to meet on the 18th of August to confirm the work I am
doing for you. Please contact me ASAP.

George

(garba@kali)-[~/usb]
$ icat Ch01InChap01.dd 15
Earl,
We need to meet on the 18th of August to confirm the work I am
doing for you. Please contact me ASAP.

George
```

The command `icat Ch01InChap01.dd 15 > letter1.txt` extracted the raw data associated with inode 15 without requiring the original filename. The command `cat letter1.txt` displayed the recovered message content from George to Earl regarding an August 18th meeting.

PART B: METADATA STRUCTURE INSPECTION (`istat`)

`istat` was executed on inode 15 to inspect metadata properties and cluster allocations.

```
(garba@kali)-[~/usb]
$ istat Ch01InChap01.dd 15
Directory Entry: 15
Not Allocated
File Attributes: File, Archive
Size: 121
Name: _ETTER1.TXT

Directory Entry Times:
Written: 2005-12-09 06:51:50 (WAT)
Accessed: 2005-12-09 00:00:00 (WAT)
Created: 2005-12-09 06:59:09 (WAT)

Sectors:
312
```

The command `istat Ch01InChap01.dd 15` confirmed the file allocation state as **Not Allocated** (deleted), listed a file size of 121 bytes, and verified that its content resides on sector **312**.

PART C: DATA UNIT EXTRACTION AND HEX DUMP (blkcat & blkls)

Raw data units were inspected directly using `blkcat` and `blkls`.

```
(garba@kali)-[~/usb]
$ blkcat Ch01InChap01.dd 312
Earl,
We need to meet on the 18th of August to confirm the work I am
doing for you. Please contact me ASAP.
George

(garba@kali)-[~/usb]
$ blkls Ch01InChap01.dd 312-312
Earl,
We need to meet on the 18th of August to confirm the work I am
doing for you. Please contact me ASAP.
George

(garba@kali)-[~/usb]
$ blkls Ch01InChap01.dd 312 | xxd
Error stat(ing) image file (raw_open: image "312" - No such file or directory)

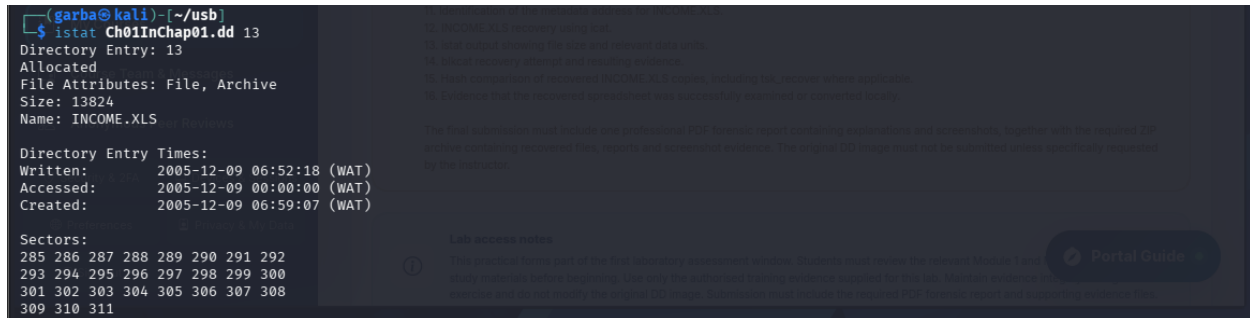
(garba@kali)-[~/usb]
$ blkcat Ch01InChap01.dd 312 | xxd
00000000: 4561 726c 2c20 0d0a 5765 206e 6565 6420  Earl, ..We need
00000010: 746f 206d 6565 7420 6f6e 2074 6865 2031  to meet on the 1
00000020: 3874 6820 6f66 2041 7567 7573 7420 746f  8th of August to
00000030: 2063 6f6e 6669 726d 2074 6865 2077 6f72  confirm the wor
00000040: 6b20 4920 616d 200d 0a64 6f69 6e67 2066  k I am ..doing f
00000050: 6f72 2079 6f75 2e20 506c 6561 7365 2063  or you. Please c
00000060: 6f6e 7461 6374 206d 6520 4153 4150 2e0d  ontact me ASAP..
00000070: 0a0d 0a47 656f 7267 6500 0000 0000 0000  ... George.....
00000080: 0000 0000 0000 0000 0000 0000 0000 0000
00000090: 0000 0000 0000 0000 0000 0000 0000 0000
000000a0: 0000 0000 0000 0000 0000 0000 0000 0000
000000b0: 0000 0000 0000 0000 0000 0000 0000 0000
000000c0: 0000 0000 0000 0000 0000 0000 0000 0000
000000d0: 0000 0000 0000 0000 0000 0000 0000 0000
000000e0: 0000 0000 0000 0000 0000 0000 0000 0000
000000f0: 0000 0000 0000 0000 0000 0000 0000 0000
00000100: 0000 0000 0000 0000 0000 0000 0000 0000
00000110: 0000 0000 0000 0000 0000 0000 0000 0000
00000120: 0000 0000 0000 0000 0000 0000 0000 0000
00000130: 0000 0000 0000 0000 0000 0000 0000 0000
00000140: 0000 0000 0000 0000 0000 0000 0000 0000
00000150: 0000 0000 0000 0000 0000 0000 0000 0000
00000160: 0000 0000 0000 0000 0000 0000 0000 0000
00000170: 0000 0000 0000 0000 0000 0000 0000 0000
```

The command `blkcat Ch01InChap01.dd 312` read sector 312 directly from the disk image. `blkls Ch01InChap01.dd 312-312` extracted unallocated space metrics for that sector. Pipe-ing `blkcat` to `xxd` displayed sector contents in hexadecimal format, confirming ASCII text and trailing zero-padding up to the sector boundary (0x1ff / 512 bytes).

TASK 4: TARGET ANALYSIS & RECOVERY OF INCOME.XLS

PART A: TARGET METADATA ANALYSIS (istat)

istat was executed specifically on Inode **13** to analyze metadata structure and sector mappings for INCOME.XLS.



```
(garba@kali) (~/.usb)
$ istat Ch01InChap01.dd 13
Directory Entry: 13
Allocated
File Attributes: File, Archive
Size: 13824
Name: INCOME.XLS

Directory Entry Times:
Written: 2005-12-09 06:52:18 (WAT)
Accessed: 2005-12-09 00:00:00 (WAT)
Created: 2005-12-09 06:59:07 (WAT)

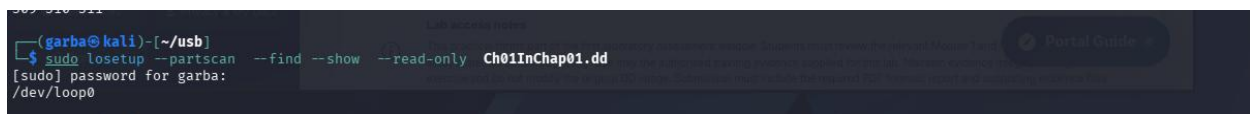
Sectors:
285 286 287 288 289 290 291 292
293 294 295 296 297 298 299 300
301 302 303 304 305 306 307 308
309 310 311
```

The command `istat Ch01InChap01.dd 13` displayed metadata properties for INCOME.XLS:

- **Directory Entry:** 13
- **Allocation Status:** Allocated
- **File Size:** 13824 bytes
- **Sectors Allocated:** 285 286 287 288 289 290 291 292 293 294 295 296 297 298 299 300 301 302 303 304 305 306 307 308 309 310 311 (A contiguous sequence of 27 sectors).

PART B: LOOP-MOUNTING RAW IMAGE (losetup)

The raw image was loop-mounted into the Linux kernel to access file system contents directly via local file management tools.

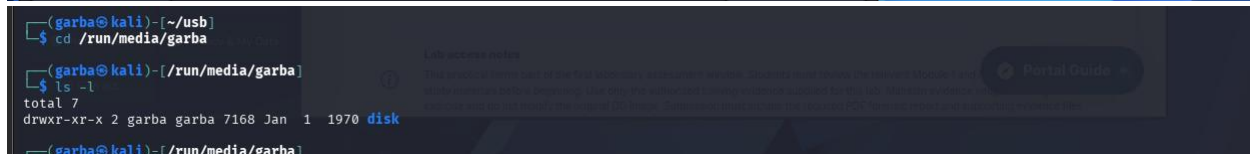
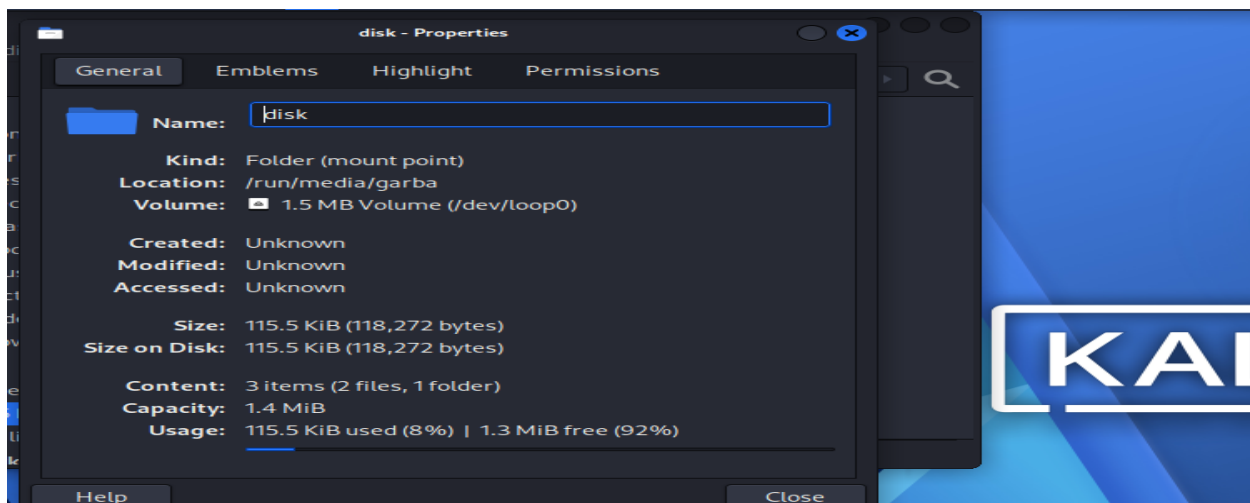
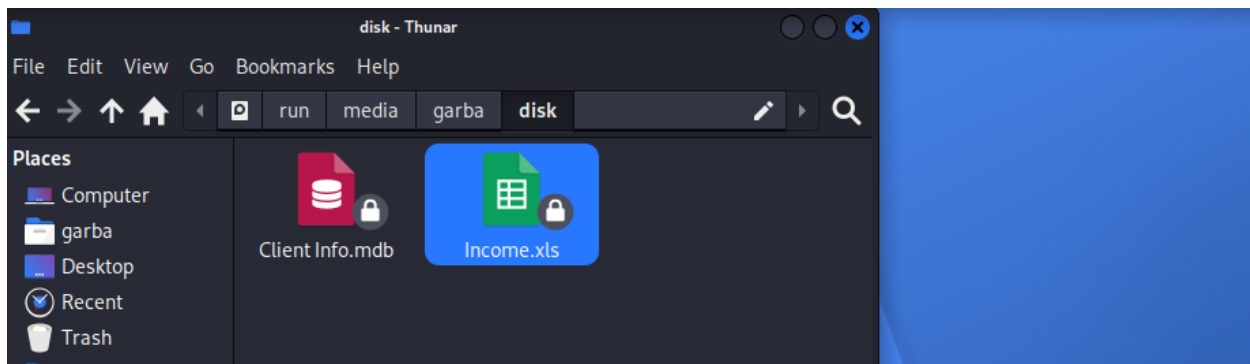


```
(garba@kali) (~/.usb)
$ sudo losetup --partscan --find --show --read-only Ch01InChap01.dd
[sudo] password for garba:
/dev/loop0
```

The command `sudo losetup --partscan --find --show --read-only Ch01InChap01.dd` created a read-only loop device assigned to `/dev/loop0` (and later `/dev/loop1` on re-execution). The `--partscan` option scanned for partitions, while `--read-only` preserved evidence integrity.

PART C: LOCAL FILE SYSTEM MOUNT & EXAMINATION

The mounted 1.5 MB volume was opened in the Thunar File Manager and terminal environment to verify local access.

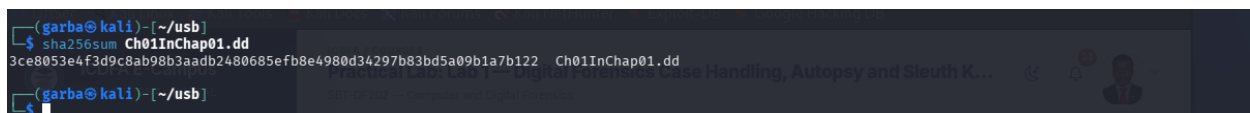


Opening the desktop volume mounted at `/run/media/garba/disk` displayed target files `Client Info.mdb` and `Income.xls`. Right-clicking properties in Thunar confirmed a total file system usage of 115.5 KiB (118,272 bytes) formatted as msdos (FAT12). Running `ls -l` in terminal confirmed directory access permissions and mounting under `/run/media/garba/disk`.

TASK 5: ADVANCED RECOVERY, HASH COMPARISON & EVIDENCE VERIFICATION

PART A: SHA-256 IMAGE INTEGRITY VERIFICATION

The SHA-256 cryptographic hash of the evidence image was calculated to fulfill evidence baseline requirements.



The command `sha256sum Ch01InChap01.dd` calculated the SHA-256 digest:
 3ce8053e4f3d9c8ab98b3aadb2480685efb8e4980d34297b83bd5a09b1a7b122.

PART B: RECURSIVE DELETED FILE LISTING (fls -r -d)

A recursive listing of deleted directory entries was generated to fulfill deliverable requirements.

```
(garba@kali)~[/usb]
$ fls -r -d Ch01InChap01.dd
r/r * 8:      Billing Letter.doc
r/r * 11:     confirmation.txt
r/r * 15:     letter1.txt
r/r * 17:     Regrets.doc
(garba@kali)~[/usb]
```

The command `fls -r -d Ch01InChap01.dd` searched recursively (-r) for deleted files (-d). Output confirmed deleted files across root directory entries:

PART C: MULTI-VECTOR RECOVERY & HASH INTEGRITY MATRIX

INCOME.XLS was recovered using three distinct forensic techniques (icat inode extraction, blkcat sector concatenation loop, and direct read-only loop mount) to perform verification hashing.

```
(garba@kali)~[/usb]
$ icat Ch01InChap01.dd 13 > income_icat.xls

(garba@kali)~[/usb]
$ md5sum income_icat.xls income_blkcat.xls /run/media/garba/disk/Income.xls
6a2e65afc5af4fc5f9da2859df134eac income_icat.xls
6a2e65afc5af4fc5f9da2859df134eac income_blkcat.xls
6a2e65afc5af4fc5f9da2859df134eac /run/media/garba/disk/Income.xls
(garba@kali)~[/usb]
```

The command `icat Ch01InChap01.dd 13 > income_icat.xls` extracted the file via inode 13. The for loop appended data units from sector 285 to 311 into `income_blkcat.xls`.